

ReBAC V3.4 — Enterprise Features

Branch: `ReBAC-V3.4` | **Builds on:** `V3.3 Compiler Enhancements`

The Problem V3.3 Left Unsolved

The authorization engine is now correct, optimized, and compiled. But real enterprise applications have requirements that go beyond graph traversal:

1. **Conditional permissions** — a doctor can view a patient record *only during their shift*
2. **Attribute-based conditions** — a patient can be discharged *only if their status is READY*
3. **Explainability** — "Why was Raj denied access?" needs a structured answer, not just `false`
4. **Multi-tenancy** — Apollo Hospital and AIIMS must have completely isolated schemas
5. **IDE support** — engineers writing `.auth` files need autocomplete and error highlighting

V3.4 Architecture

1. Caveats — Conditional Permissions

New DSL syntax:

```
permission view = doctor_of IF shift_active
permission view = admin OR (doctor_of IF shift_active)
```

CaveatEvaluator: Registry-based conditional evaluation engine.

```
caveatEvaluator.register("shift_active", (ctx) =>
  ctx.attributes?.shift_active === true);
caveatEvaluator.register("emergency_mode", (ctx) =>
  ctx.attributes?.emergency === true);
caveatEvaluator.register("same_department", (ctx) =>
  ctx.attributes?.user?.department ===
```

```
ctx.attributes?.resource?.department
);
```

Caveats are named boolean conditions evaluated at **runtime** against request-time context. This is Zanzibar's own caveat concept, introduced in 2023 as a first-class feature in SpiceDB and OpenFGA.

New AST node:

```
interface CaveatExpressionNode {
  nodeType: "CaveatExpression";
  relation: RelationNode;    // the base relation
  caveat: string;            // name of the registered caveat
  function
}
```

2. ABAC — Attribute-Based Conditions

New DSL syntax:

```
permission discharge = doctor_of AND patient.status == "READY"
permission edit = admin AND user.department == patient.department
```

ABACEvaluator: Resolves attribute paths and evaluates comparison operators.

```
// Runtime: pass attributes at checkPermission time
checkPermission({
  userId: raj.id,
  resourceId: patient.id,
  permission: "discharge",
  attributes: {
    patient: { status: "READY" },
    user: { department: "ICU" }
  }
})
```

Supported operators: `==` , `!=` , `<` , `>` , `<=` , `>=`

New AST nodes:

```
interface AttributePathNode { object: string; field: string; }
interface AttributeConditionNode {
  left: AttributePathNode | string;
  operator: ComparisonOperator;
  right: AttributePathNode | string;
}
```

3. Explain Tree — Structured Decision Trace

Unlike V2.3's static Explain Plan (what *will* run) and V3.3's profiler (raw metrics), the Explain Tree provides a **structured, human-readable decision trace** showing exactly *why* access was granted or denied:

```
EXPLAIN patient.view — Result: ALLOW
├─ ✓ patient.view
│   └─ ✓ OR [0.91ms]
│       └─ ✓ patient.medical_staff
│           └─ ✓ Direct: doctor_of [0.77ms]
```

This mirrors PostgreSQL's `EXPLAIN ANALYZE` output with tree-structured per-node timing and result annotations — and is what every production authorization system's "Why was I denied?" feature looks like.

4. Multi-Tenant Compilation — TenantRegistry

Each hospital (tenant) has a completely **isolated compiled schema**:

```
globalTenantRegistry.register({ tenantId: "apollo", schemaPath:
  "apollo.auth" });
globalTenantRegistry.register({ tenantId: "aiims", schemaPath:
  "aiims.auth" });

// Apollo uses vip_doctor_of; AIIMS uses govt_doctor_of — fully
isolated
checkPermission({ userId, resourceId, permission, tenantId:
  "apollo" })
```

Each tenant gets:

- Its own `CompilationCache` instance

- Its own compiled `RebacSchema`
- Zero shared mutable state between tenants

5. IDE / Language Server Protocol (LSP)

The ADSL Language Server implements the full LSP protocol:

Capability	Description
textDocument/diagnostics	Real-time error highlighting from ADSL linter
textDocument/completion	Keyword + declared resource/relation suggestions
textDocument/hover	Documentation on hover over resource, IF, relation
textDocument/documentSymbol	Outline: all resources, relations, permissions

Critical Review

✔ What V3.4 Gets Right

1. **Caveats are a real Zanzibar concept.** SpiceDB 1.0 introduced caveats in 2023. OpenFGA 1.0 introduced `condition` blocks. V3.4's implementation is semantically faithful — a caveat is a named, runtime-evaluated boolean condition.
2. **ABAC over ReBAC** is the industry direction. AWS Cedar explicitly combines RBAC + ABAC + ReBAC. Pure ReBAC is insufficient for enterprise use cases that need attribute-based conditions.
3. **Explain Tree is the correct output format** for a production "why denied?" feature. AWS Cedar's `authorize_with_errors` API returns exactly this kind of structured explanation.
4. **Multi-tenant isolation via separate compiler instances** is the correct architecture — no shared state between tenants prevents privilege escalation across tenant boundaries.

⚠ What V3.4 Is Still Missing

Missing: Caveat compile-time validation Caveats are registered at runtime. The DSL cannot verify at compile time that `IF shift_active` refers to a registered caveat — it would only fail at runtime. SpiceDB validates caveats against a caveat schema at schema-compile time.

Missing: ABAC attribute schema There is no DSL for declaring what attributes a resource has. `patient.status` is a runtime attribute with no compile-time type declaration. This means `patient.status == "READY"` (typo) passes silently until runtime.

Missing: Write token / consistency guarantee After writing a tuple, the next `checkPermission` might read from a replica that hasn't received the write yet. Zanzibar's zookies / SpiceDB's ZedTokens solve this with consistency tokens.

Missing: Audit log Enterprise systems require an immutable audit trail: who checked what permission, when, and what the decision was. V3.4 has an Explain Tree per request but no persistent audit log.

Comparison: V3.4 vs Production Authorization Systems

Feature	V3.4	SpiceDB	OpenFGA	AWS Cedar	OPA
Graph-based ReBAC	✓	✓	✓	Partial	✓
Custom DSL/schema	✓ ADSL	✓ zed	✓	✓ Cedar	✓ Rego
Caveats	✓	✓	✓ conditions	Via attributes	Via rules
ABAC	✓	Partial	✓	✓ full	✓ full
Explain Tree	✓	✓	✓	✓	✓
Multi-tenancy	✓	✓ namespaces	✓ stores	✓ accounts	✓
LSP server	✓	✓	✓	✓	✓
Consistency tokens	✗	✓ ZedTokens	✓	N/A	N/A
Audit log	✗	✓	✓	✓	✓
Distributed	✗	✓	✓	AWS- managed	✓

Summary

V3.4 marks the completion of the educational ReBAC platform, adding four enterprise-grade features: Caveats (runtime conditional permissions matching Zanzibar's caveat model), ABAC

(attribute-based conditions bridging ReBAC and ABAC paradigms), Explain Tree (structured per-decision audit trace), Multi-Tenant Compilation (fully isolated schema instances), and an IDE Language Server. The system is now feature-comparable to early SpiceDB or OpenFGA releases. The remaining gaps (consistency tokens, attribute schema validation, audit persistence, distributed architecture) represent the next frontier — V4.x territory.